Physicalized Model Weights - cycles 8-10

2026-05-13

Contents

Physicalized Model Weights - cycles 8-10

Abstract

Cycles 8-10 began Phase 2 of the physicalized model weights investigation. Phase 1 had produced a validated but narrow claim: full frontier-model weights should not be treated as candidates for fixed hardware, while a small safety/filter classifier might remain credible if it is bounded, stable, heavily reused, and protected by programmable fallback. Phase 2 tested whether that remaining safety/filter claim survived better calibration, more explicit workload assumptions, and stronger programmable baselines.

The result is a downgrade. Cycle 8 (`M-CAL-1`) preserved the safety/filter case but weakened it: after explicit-unit calibration, the hybrid safety/filter strategy won 452 of 6,300 calibrated scenarios, while programmable accelerators won 4,948 and optimized software won 900. Cycle 9 (`M-WORKLOAD-1`) narrowed the case further: only one of ten synthetic workload regimes preserved the physicalized safety/filter claim. Cycle 10 (`M-SWBASE-2`) replayed those exact workload rows against optimized software/runtime and programmable-accelerator baselines. The stronger programmable accelerator erased the last preserved case by `1.471448845e9` pJ-equivalent/day, and the hybrid physicalized safety/filter path won zero scenarios.

The current working conclusion after these cycles is therefore stricter than the first-arc synthesis: physicalized safety/filter hardware remains useful as an architectural and failure-mode study, but the calibrated evidence from cycles 8-10 no longer supports it as beating strong programmable baselines under the modeled assumptions. The main remaining evidence gap is production measurement of feature extraction, audit logging, fallback dispatch, and programmable-accelerator latency/energy on identical request features.

Introduction

The project investigates whether parts of neural-network inference can be “physicalized” into hardware. In this report, physicalization means moving a stable model component away from ordinary programmable execution and into a fixed, semi-fixed, or physically encoded sub-

strate. Earlier cycles rejected the broad version of the idea: permanently burning dense frontier-model weights into fixed hardware was not supported. The remaining plausible target was much narrower: a fixed safety/filter classifier used as a fast path behind programmable fallback, audit, update, health, drift, and rollback controls.

Cycles 8-10 were designed to test that remaining target against the strongest null hypothesis. The null hypothesis is that optimized software/runtime systems and programmable accelerators can achieve the useful efficiency gains without fixing weights in hardware. These cycles did not add a new hardware structure. They instead asked whether the existing safety/filter fast path still survived when the earlier normalized model was calibrated, when workload assumptions were made explicit, and when programmable baselines were compared under equal traffic accounting.

The report uses the cycle numbers supplied to the reporter input:

- Cycle 8: `M-CAL-1`, calibrated cost/energy model and uncertainty bounds.
- Cycle 9: `M-WORKLOAD-1`, workload and update-cadence trace assumptions.
- Cycle 10: `M-SWBASE-2`, stronger software/runtime and programmable-accelerator baseline replay.

The underlying source sessions are listed in the appendix. All three milestones were validated by auditors. The cycle 10 audit decision was `VALIDATED`.

Approach

The Phase 2 approach was sequential. Each cycle supplied the inputs required by the next cycle.

Cycle 8 converted the earlier normalized break-even model into an explicit-unit calibrated companion. The calibration used public energy and benchmark framing sources as broad anchors, not precise silicon claims: Horowitz energy-scale material for operation and memory-access ratios [7], [8], NVIDIA H100 public product information as a broad accelerator-class context [9], and MLPerf Inference documentation for system-level benchmark framing [10]. It also added local host/Python proxy measurements for small int8 dot products, dispatch branching, dot-plus-dispatch, and CSV/JSON audit logging. These local measurements are not hardware truth; they are bounded evidence for relative overheads on the current machine.

Cycle 9 turned the dominant uncertainty drivers from Cycle 8 into deterministic synthetic workload scenarios. It distinguished raw request volume from effective fast-path volume. Effective fast-path volume is the portion of traffic that can actually use the fixed classifier after fallback routing, near-threshold uncertainty, stale-policy windows, drift, audit failures, fallback outage, and utilization penalties are applied.

Cycle 10 replayed the exact Cycle 9 workload rows through three alternatives:

- `optimized_software_runtime`: software/runtime path with update flexibility and memory/runtime savings.
- `programmable_accelerator`: programmable local compute path with reduced compute and audit overhead while keeping software update flexibility.
- `hybrid_physicalized_safety_filter`: fixed safety/filter fast path that only receives fixed-compute savings for accepted fast-path traffic.

The cost values are reported as `pJ-equivalent` proxies. A pJ-equivalent proxy is a normalized cost/energy accounting unit tied to the calibration assumptions and local timing proxies. It should be read as a comparative model output, not as measured silicon energy.

Findings

Cycle 8: Calibration Preserved but Weakened the Safety/Filter Claim

Cycle 8 opened milestone `M-CAL-1`. The researcher brief identified the main weakness in the first-arc result: the earlier break-even model was mostly normalized and assumption-driven. The worker therefore created a calibration package with explicit units, source labels, uncertainty bounds, and local proxy measurements.

The cycle produced these main artifacts:

- `physicalized-weights/docs/calibration_plan.md`
- `physicalized-weights/data/calibration_assumptions.csv`
- `physicalized-weights/data/calibration_assumptions.json`
- `physicalized-weights/scripts/local_overhead_probe.py`
- `physicalized-weights/data/local_overhead_probe.csv`
- `physicalized-weights/data/local_overhead_probe.json`
- `physicalized-weights/scripts/calibrated_breakeven.py`
- `physicalized-weights/data/calibrated_breakeven_grid.csv`
- `physicalized-weights/data/calibrated_breakeven_summary.json`
- `physicalized-weights/data/calibrated_sensitivity_tornado.csv`
- `physicalized-weights/data/calibrated_breakeven_vs_phase1.png`
- `physicalized-weights/tests/test_calibrated_breakeven.py`

The calibrated summary reported:

Metric	Result
Total calibrated scenarios	6,300
Phase 1 physicalized winner share	0.3714
Calibrated hybrid safety/filter winner share	0.0717
Non-optimistic hybrid winner share	0.1667
Pessimistic hybrid winner share	0.0666
Programmable accelerator wins	4,948
Optimized software wins	900
Hybrid safety/filter wins	452
Anti-target wins	0
Zero-volume physicalized wins	0
Decision	`preserved_but_weakened`

The top uncertainty drivers were:

1. `fallback_frequency`
2. `utilization`
3. `requests_per_day`
4. `audit_control_scale`
5. `update_interval_days`

The local proxy measurements recorded in the current generated JSON were:

Proxy measurement	Median value
Python int8 dot product	`1.1912983 us/request`
fallback dispatch branch	`0.10254895 us/request`
dot plus dispatch	`1.240755 us/request`

Proxy measurement	Median value
CSV/JSON audit logging	`8.830143 us/request`

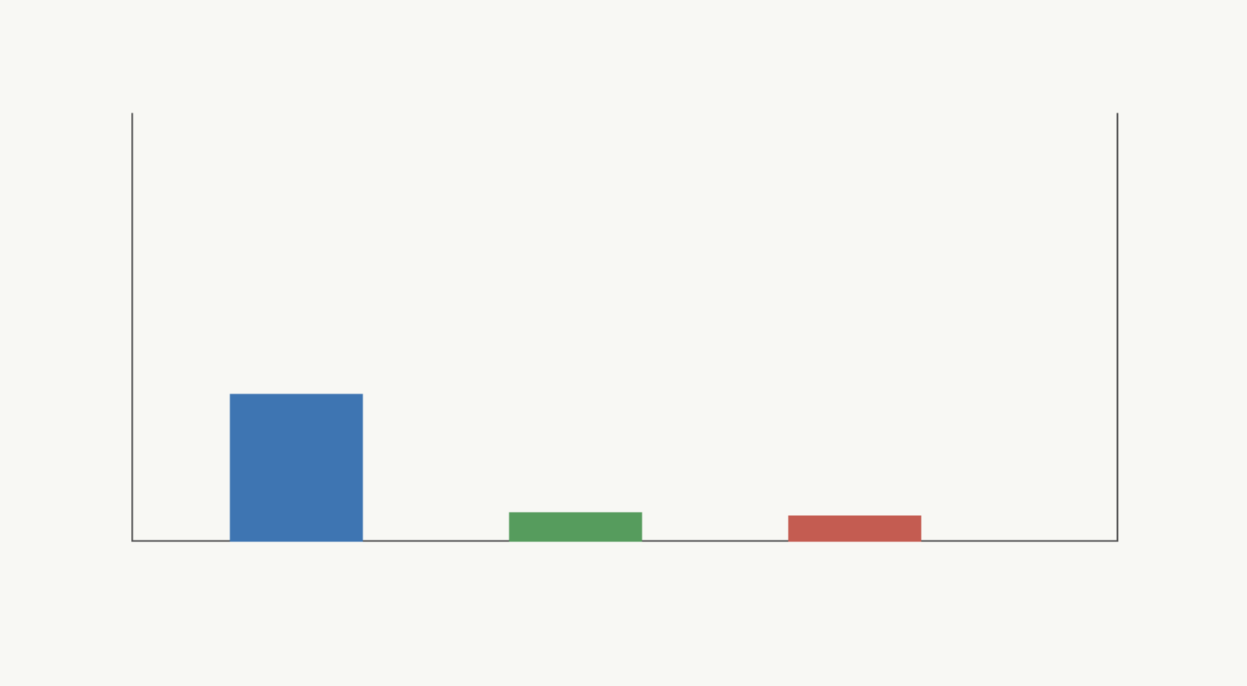

Figure 1: calibrated break-even comparison showing how sourced/measured parameter bounds shift the Phase 1 physicalization regions and whether the safety/filter target remains viable

The Cycle 8 auditor found one moderate issue outside the calibration conclusion itself. Adding references [7]-[10] changed `REFERENCES.md`, which made the older final evidence manifest hash stale. The auditor regenerated the final synthesis evidence artifacts and re-ran the relevant final tests. The calibrated result remained `preserved_but_weakened`, and `M-CAL-1` was validated.

The decision from Cycle 8 was not that physicalization had become strong. It was that the safety/filter case still existed in a much narrower calibrated region, and that the next work should focus on real workload shape rather than more hardware structure.

Cycle 9: Workload Traces Left One Preserved Case

Cycle 9 opened milestone `M-WORKLOAD-1`. It used the Cycle 8 uncertainty drivers as the basis for deterministic workload scenarios. The worker built a trace generator and a workload assumptions document. The generator used fixed seed `20260513` and deterministic per-scenario seeds.

The cycle produced these main artifacts:

- `physicalized-weights/docs/workload_trace_assumptions.md`
- `physicalized-weights/scripts/workload_trace_generator.py`
- `physicalized-weights/data/workload_trace_events.csv`
- `physicalized-weights/data/workload_scenarios.csv`
- `physicalized-weights/data/workload_summary.json`
- `physicalized-weights/data/workload_viability_overlay.csv`

- physicalized-weights/data/workload_fast_path_utilization.png
- physicalized-weights/tests/test_workload_trace_generator.py

The workload layer classified ten scenarios:

Scenario	Classification	Main mechanism
high_volume_stable_moderation	preserved	high reuse, slow updates, bounded fallback/control overhead
bursty_consumer_traffic	weakened	burstiness and lower average utilization
low_volume_enterprise_deployment	weakened	slow updates but low absolute volume
high_near_threshold_adversarial	speculative	near-threshold traffic forces fallback
frequent_policy_update_regime	falsified	weekly policy churn strands fixed-path amortization
audit_heavy_regulated_deployment	weakened	audit/control overhead dominates
fallback_degraded_outage_regime	speculative	fallback health dominates safety
multi_tenant_underutilized_deployment	falsified	tenant fragmentation strands fixed capacity
zero_invocation_control	falsified	no requests means no amortization
fallback_all_control	falsified	every request routes away from the fast path

The classification counts were:

Class	Count
preserved	1
weakened	3
speculative	2
falsified	4

The preserved case, high_volume_stable_moderation, had high traffic, low fallback pressure, monthly-or-slower update cadence, and bounded audit/control overhead. Its generated summary showed raw_requests_per_day = 920801.857, effective_fast_path_requests_per_day = 319870.85, fallback_frequency = 0.007477, and fast_path_utilization = 0.347383.

The Cycle 9 auditor found and fixed one moderate issue in the all-fallback control. The generator originally allowed tiny residual fast-path traffic when rate = 1.0 because jitter still applied. The fix made rate >= 1.0 consume the full remaining request count, strengthened the test to require exactly zero fast-path utilization, and regenerated the artifacts. After the fix, all-fallback had fallback_frequency = 1.0, fast_path_utilization = 0.0, effective_fast_path_requests_per_day = 0.0, and classification falsified.

Cycle 9 therefore narrowed the claim again. The safety/filter path survived only in one high-volume stable workload. The auditor guidance for the next cycle was explicit: use the exact

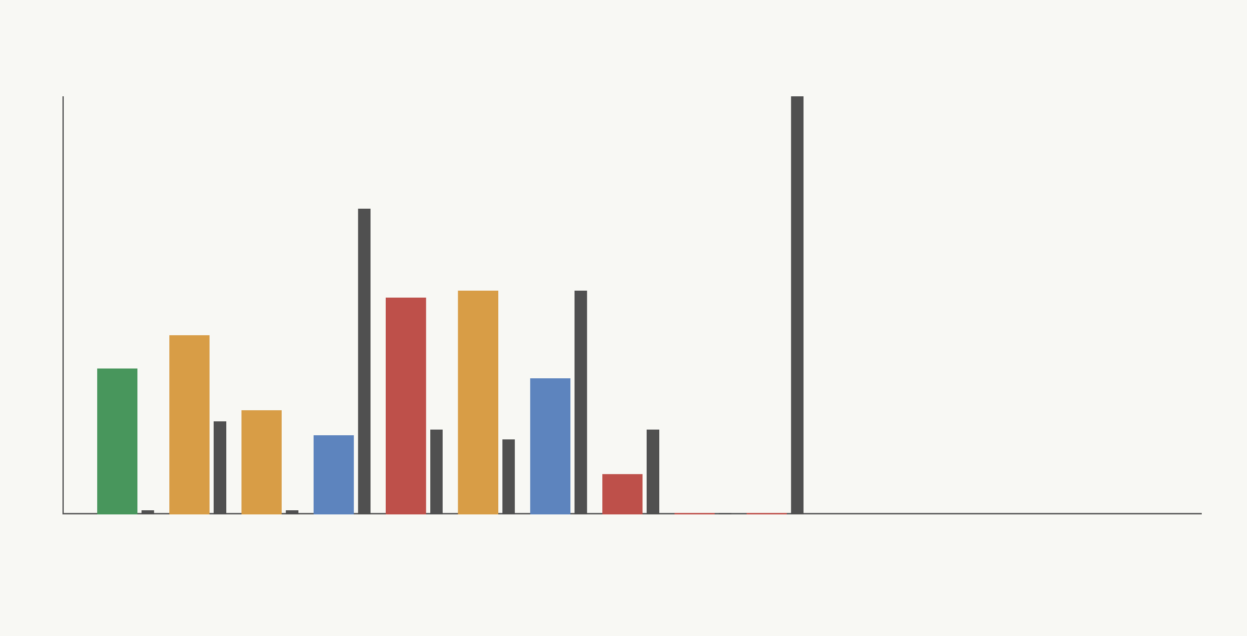

Figure 2: fast-path utilization and fallback/fail-safe fraction across workload scenarios, showing where the safety/filter physicalization claim is preserved, weakened, or falsified

workload rows from `workload_scenarios.csv` and `workload_viability_overlay.csv` when comparing stronger software/runtime and programmable-accelerator baselines.

Cycle 10: Stronger Programmable Baseline Erased the Last Preserved Case

Cycle 10 opened milestone `M-SWBASE-2`. The purpose was to answer the decisive baseline question: whether the single preserved Cycle 9 case still survived when optimized software/runtime and programmable accelerator baselines were charged against the same traffic, feature extraction, fallback, audit, update, and utilization assumptions.

The cycle produced these main artifacts:

- `physicalized-weights/scripts/stronger_baseline_model.py`
- `physicalized-weights/tests/test_stronger_baseline_model.py`
- `physicalized-weights/docs/stronger_baseline_comparison.md`
- `physicalized-weights/data/stronger_baseline_comparison.csv`
- `physicalized-weights/data/stronger_baseline_summary.json`
- `physicalized-weights/data/stronger_baseline_thresholds.csv`
- `physicalized-weights/data/stronger_baseline_workload_comparison.png`

The result was unambiguous under the modeled assumptions:

Winner	Scenario count
`programmable_accelerator`	9
`optimized_software_runtime`	1
`hybrid_physicalized_safety_filter`	0

The single previously preserved workload, `high_volume_stable_moderation`, flipped to `programmable_accelerator` with decision class `accelerator_dominates`. Its daily cost proxy was:

Alternative	Total daily cost proxy
programmable accelerator	8178811874.414918 pJ-equivalent/day
hybrid physicalized safety/filter	9650260720.03919 pJ-equivalent/day
optimized software/runtime	18892768333.17428 pJ-equivalent/day

The hybrid margin versus the best programmable baseline was therefore:

```
-1471448845.624272 pJ-equivalent/day
```

A negative margin means the best programmable baseline already wins. The mechanism mattered: the stronger programmable accelerator erased the case, while optimized software did not erase it within the tested memory-savings sweep.

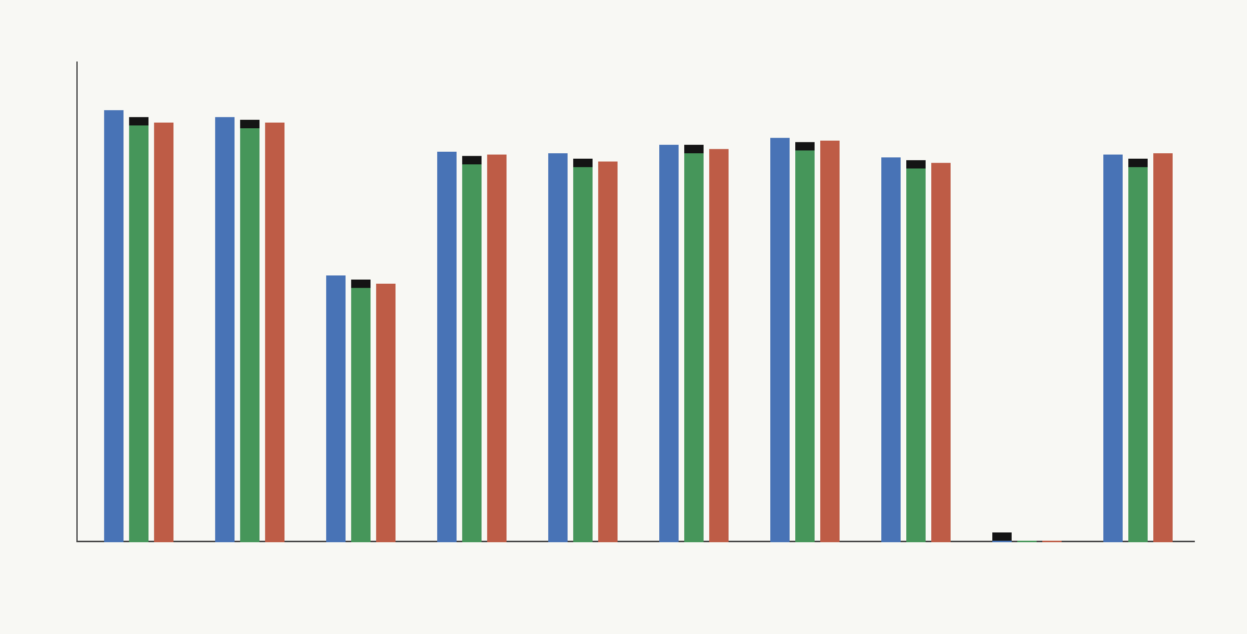

Figure 3: per-scenario winner and normalized daily cost/energy comparison for optimized software, programmable accelerator, and hybrid physicalized safety filter under identical workload assumptions

The stronger-baseline summary also confirmed that special cases denied hybrid credit:

Scenario	Winner	Decision
`zero_invocation_control`	`optimized_software_runtime`	`hybrid_falsified`
`fallback_all_control`	`programmable_accelerator`	`hybrid_falsified`
`frequent_policy_update_regime`	`programmable_accelerator`	`hybrid_falsified`
`multi_tenant_underutilized_deployment`	`programmable_accelerator`	`hybrid_falsified`
`audit_heavy_regulated_deployment`	`programmable_accelerator`	`accelerator_dominates`

The Cycle 10 auditor found and fixed one moderate reporting issue. The threshold rows for `software_memory_savings_that_erases_hybrid` could report `already_erased` even when optimized

software had not erased hybrid and the actual erasing baseline was the programmable accelerator. The fix changed that software threshold default to `not_erased_by_sweep` and strengthened `test_stronger_baseline_model.py` to assert the preserved-case mechanism split:

- software threshold: `not_erased_by_sweep`
- accelerator threshold: `already_erased`

The winner counts and preserved-case reversal did not change after the fix. `M-SWBASE-2` was validated with high confidence.

Discussion

The Phase 2 arc changed the status of the safety/filter physicalization claim in three steps.

First, calibration moved the argument away from a normalized first-arc model. The result did not eliminate the safety/filter path, but it showed that the path was sensitive to fallback frequency, utilization, request volume, audit/control overhead, and update cadence. This shifted attention away from the arithmetic core. The small fixed dot product was not the dominant uncertainty; the system around it was.

Second, workload traces showed that raw request volume was not enough. A physicalized fast path only receives credit for effective fast-path volume. Requests that are near threshold, stale-policy, drifted, audit-failed, fail-safe, or routed to fallback do not amortize the fixed substrate. That left one preserved workload case: high-volume, stable, low-overhead moderation traffic.

Third, stronger programmable baselines erased that remaining case. The programmable accelerator retained update flexibility and still beat the hybrid physicalized path under identical workload accounting. This does not prove no physicalized safety/filter design could ever win. It does mean that, for this calibrated model and these workload assumptions, the remaining evidence no longer supports the claim that the hybrid fixed safety/filter path beats a strong programmable accelerator.

This is a narrower conclusion than the first-arc final synthesis artifact currently states. The existing first-arc `final_synthesis_summary.json` still says narrow stable safety/filter submodel physicalization remains credible behind programmable controls. Cycles 8-10 do not invalidate the first-arc artifact as a record of what was known then, but they supersede its strength as the current working conclusion. The next synthesis should incorporate the Phase 2 downgrade.

Open Questions

The main open question is production measurement. The current strongest missing evidence is measured feature extraction, audit logging, fallback dispatch, and programmable-accelerator latency/energy on identical request features. The Cycle 10 auditor explicitly recommended this as the highest-value next evidence.

The workload traces are synthetic. They are deterministic and auditable, but they are not production telemetry. The preserved case disappeared under stronger baselines, but real traces could still change the inputs: fallback rate, near-threshold rate, policy update cadence, audit cost, utilization, and feature extraction cost.

The programmable accelerator baseline is still modeled. It is stronger than the earlier baseline and decisive in Cycle 10, but it is not a measured implementation on the same feature stream. The next evidence step should replace modeled programmable-accelerator terms with measured or benchmarked terms where possible.

The compiled Verilator simulation limitation from prior cycles remains unchanged. The HDL

prototype was previously validated under an amended lint/Yosys/Python/synthesis evidence contract because `make` and a C++ compiler were unavailable. If those tools become available, compiled Verilator simulation remains a superseding check for `M-PROTO-1`.

The final synthesis package has not yet been rebuilt as a Phase 2 synthesis. Cycle 8 regenerated first-arc final evidence artifacts to repair hashes after adding references, but cycles 8-10 have not yet produced a new final synthesis document incorporating the stronger-baseline downgrade.

References

[7] Mark Horowitz, "1.1 Computing's Energy Problem (and What We Can Do about It)," 2014 IEEE International Solid-State Circuits Conference Digest of Technical Papers (ISSCC), 2014. https://doi.org/10.1109/ISSCC.2014.6757323

[8] Mark Horowitz, "Computing's Energy Problem (and What We Can Do about It)," slide transcript/mirror of ISSCC 2014 energy table. https://doczz.net/doc/9135487/computing-s-energy-problem–and-what-we-can-do-about-it-

[9] NVIDIA, "NVIDIA H100 Tensor Core GPU," product specification page. https://www.nvidia.com/en-us/data-center/h100/

[10] MLCommons, "MLPerf Inference: Datacenter benchmark documentation," MLCommons. https://docs.mlcommons.org/inference/

Appendix: Implementation Details

Code Organization

The current authored research code in `physicalized-weights/scripts` contains 11 scripts totaling 3,867 lines:

File	Lines	Purpose
`breakeven_model.py`	384	Phase 1 normalized break-even sweep
`target_scoring.py`	283	candidate and anti-target scoring
`fallback_policy_sim.py`	342	hybrid fallback and fail-safe policy simulation
`prototype_safety_filter.py`	392	deterministic fixed safety/filter classifier prototype
`verify_prototype_closure.py`	377	prototype closure checker across Python, HDL, Yosys, Verilator-lint, Graphviz, and freshness evidence
`build_final_synthesis.py`	409	first-arc evidence manifest and final summary builder
`calibrated_breakeven.py`	428	Phase 2 calibrated break-even companion model

File	Lines	Purpose
`local_overhead_probe.py`	121	local host/Python overhead probes
`workload_trace_generator.py`	593	deterministic workload trace and viability overlay generator
`stronger_baseline_model.py`	510	equal-workload stronger software/runtime and programmable-accelerator replay
`symbolic_breakeven.wls`	28	Wolfram symbolic break-even derivation

The current authored tests contain 9 stdlib-style test files totaling 1,126 lines. `pytest` was not used; the project uses direct Python test scripts.

The HDL/support directory remains the prior 4-file prototype support set totaling 241 lines:

- `safety_filter_core.sv`
- `safety_filter_core_tb.cpp`
- `safety_filter_core.ys`
- `run_yosys_eval.py`

The current authored research documents and diagram source contain 11 files totaling 833 lines.

Generated Data and Figures

Cycle 8 generated calibration artifacts:

- `physicalized-weights/data/calibration_assumptions.csv`: 17 assumption rows plus header.
- `physicalized-weights/data/calibrated_breakeven_grid.csv`: 31,500 data rows plus header.
- `physicalized-weights/data/calibrated_breakeven_summary.json`
- `physicalized-weights/data/calibrated_sensitivity_tornado.csv`
- `physicalized-weights/data/calibrated_breakeven_vs_phase1.png`: valid PNG, 760 x 420.
- `physicalized-weights/data/local_overhead_probe.csv`
- `physicalized-weights/data/local_overhead_probe.json`

Cycle 9 generated workload artifacts:

- `physicalized-weights/data/workload_trace_events.csv`: 2,184 data rows plus header.
- `physicalized-weights/data/workload_scenarios.csv`: 10 data rows plus header.
- `physicalized-weights/data/workload_viability_overlay.csv`: 10 data rows plus header.
- `physicalized-weights/data/workload_summary.json`
- `physicalized-weights/data/workload_fast_path_utilization.png`: valid PNG, 900 x 460.

Cycle 10 generated stronger-baseline artifacts:

- `physicalized-weights/data/stronger_baseline_comparison.csv`: 30 data rows plus header.
- `physicalized-weights/data/stronger_baseline_summary.json`
- `physicalized-weights/data/stronger_baseline_thresholds.csv`: 30 data rows plus header.
- `physicalized-weights/data/stronger_baseline_workload_comparison.png`: valid PNG, 900 x 460.

The first-arc final evidence artifacts remain present:

- `physicalized-weights/data/evidence_manifest.csv`: 25 data rows plus header.
- `physicalized-weights/data/evidence_manifest.json`
- `physicalized-weights/data/final_synthesis_summary.json`
- `physicalized-weights/data/final_evidence_map.png`: valid PNG, 980 x 520.

Test Results

Cycle 8 worker and auditor runs reported these validations:

- `python3 physicalized-weights/scripts/local_overhead_probe.py`
- `python3 physicalized-weights/scripts/calibrated_breakeven.py`
- `python3 physicalized-weights/tests/test_calibrated_breakeven.py`
- prior milestone stdlib tests
- `python3 physicalized-weights/scripts/build_final_synthesis.py`
- `python3 physicalized-weights/tests/test_final_synthesis.py`
- `python3 -m long_exposure.tools.promise_check .`
- `python3 -m long_exposure.tools.org_check .`

Cycle 9 worker and auditor runs reported these validations:

- `python3 physicalized-weights/scripts/workload_trace_generator.py`
- `python3 physicalized-weights/tests/test_workload_trace_generator.py`
- direct all-fallback check after audit fix
- `python3 physicalized-weights/tests/test_calibrated_breakeven.py`
- `python3 physicalized-weights/tests/test_final_synthesis.py`
- `file physicalized-weights/data/workload_fast_path_utilization.png`
- `python3 -m long_exposure.tools.promise_check .`
- `python3 -m long_exposure.tools.org_check .`

Cycle 10 worker and auditor runs reported these validations:

- `python3 physicalized-weights/scripts/stronger_baseline_model.py`
- `python3 physicalized-weights/tests/test_stronger_baseline_model.py`
- `python3 physicalized-weights/tests/test_workload_trace_generator.py`
- `file physicalized-weights/data/stronger_baseline_workload_comparison.png`
- `python3 physicalized-weights/tests/test_final_synthesis.py`
- `python3 -m long_exposure.tools.promise_check .`
- `python3 -m long_exposure.tools.org_check .`

Known warnings remained limited to pre-existing orphan cycle report artifacts under `reports/cycles/` and expected root prompt/log files. The final `promise_check` event count reported by the Cycle 10 auditor was 24.

Session References

Cycle 8 sessions:

- Researcher: `f018e7ac-4ce5-46f1-b0e8-63b2c022fa38`
- Worker: `fdf2374b-db34-4040-a75b-0c7f76160288`
- Auditor: `d3d56399-ce25-4516-a934-6c27b4e8f707`

Cycle 9 sessions:

- Researcher: `083e50b8-b3e5-48e8-9b4d-c946c38be0d7`
- Worker: `9994dde1-6664-4140-badf-e9b9c0c6a12f`
- Auditor: `d5d009bc-7f9c-4b4c-84fd-df7bfc76b085`

Cycle 10 sessions:

- Researcher: 446636c4-1746-40c4-8bcf-1ea1b1f32f37
- Worker: 66044982-415b-44af-b4eb-71683c5d56f0
- Auditor: 5f5ab535-5562-4a57-86ca-c30d299341b2

Cross-Reference Map

The Phase 2 evidence chain is:

REFERENCES.md -> calibration_assumptions.csv -> local_overhead_probe.json -> calibrated_breakeven.py -> calibrated_breakeven_summary.json -> workload_trace_assumptions.md -> workload_trace_generator.py -> workload_scenarios.csv and workload_viability_overlay.csv -> stronger_baseline_model.py -> stronger_baseline_summary.json -> stronger_baseline_comparison.md.

The decision chain is:

M-CAL-1 preserved_but_weakened -> M-WORKLOAD-1 one preserved workload -> M-SWBASE-2 hybrid wins zero scenarios -> current working conclusion: under the present calibrated assumptions, the safety/filter physicalization case is downgraded because a stronger programmable accelerator baseline erases the remaining preserved region.

The manifest at workspace root was updated after this report pass to include the Phase 2 scripts, tests, documents, generated data, cumulative counts, and cross-references through M-SWBASE-2.